

PROGRAMAÇÃO / DO ZERO À PRÁTICA

Seu primeiro programa.

Python

BÁSICO

- 01 Entenda a lógica.
- 02 Escreva seu código.
- 03 Construa um projeto.

UM GUIA PATODEVS

Conceitos claros, exemplos de código
e um projeto para aprender fazendo.

.py

Seu caminho de aprendizado

Este guia ensina as primeiras bases de Python. Você vai escrever programas, receber informações, fazer cálculos e organizar tarefas. Não precisa saber programar: comece pelos exemplos pequenos e avance quando conseguir explicar o resultado.

Como estudar: digite o exemplo, execute e mude uma parte. Antes de rodar novamente, tente prever o que acontecerá. Os números à esquerda dos blocos servem para leitura: não copie esses números para o arquivo.

Sumário

03 — O que um programa faz

04 — Prepare o ambiente

05-08 — Sintaxe, dados, entrada e operadores

09-11 — Decisões e repetições

12-14 — Listas, dicionários e funções

15 — Entender e tratar erros

16-19 — Projeto: organizador de tarefas

20-21 — Exercícios e respostas

22 — Glossário e referências

23 — Encerramento

Autoria editorial: PatoDevs. Edição introdutória, 2026. Exemplos para Python 3, sem bibliotecas externas. Cada bloco é independente, exceto as três partes do projeto final, que formam um único arquivo.

O que um programa faz

Um programa é uma sequência de instruções. Pense em calcular a média de duas notas: você recebe os valores, soma, divide por dois e apresenta o resultado. Essa sequência é um algoritmo. Python é uma das linguagens que permitem escrever essas instruções.

Entrada → processamento → saída

Entrada: informações recebidas, como um nome ou uma nota.

Processamento: operações e decisões sobre essas informações.

Saída: resultado apresentado, como uma mensagem no terminal.

```
01 | nota_1 = 7
02 | nota_2 = 9
03 | media = (nota_1 + nota_2) / 2
04 | print(media)
```

Resultado: 8.0. A função `print()` mostra informações na tela. A média foi calculada com os valores guardados nos nomes `nota_1` e `nota_2`; você aprenderá sobre esses nomes na página 6.

Python e o terminal

Python é usado em automações, análise de dados, aplicações e muitas outras tarefas. Aqui, vamos trabalhar no terminal: uma janela em que você digita comandos e lê respostas. Uma interface simples ajuda a observar a lógica do programa.

Em um arquivo, as instruções normalmente são executadas em ordem. Condições escolhem caminhos; repetições executam trechos várias vezes. Funções organizam instruções para reutilizá-las.

PRATIQUE

Descreva, sem código, os passos para calcular o dobro de um número. Identifique o que entra, o que é calculado e o que aparece como resposta.

Prepare seu primeiro arquivo

Use um computador, um editor de texto ou código e uma instalação de Python 3. Baixe Python pelo site oficial python.org/downloads e siga as instruções de instalação para seu sistema. No macOS e no Linux, o comando costuma ser python3.

Confira a instalação

No Windows, abra um terminal e tente o comando abaixo. Se py não estiver disponível, tente python --version. Em outros sistemas, tente python3 --version. Use nas próximas etapas o comando que mostrou uma versão de Python 3.

```
01 | py --version
```

Crie uma pasta primeiro-python. Dentro dela, salve um arquivo chamado app.py. Confira que o nome não ficou app.py.txt. O arquivo deve conter apenas esta instrução:

```
01 | print("Olá, PatoDevs!")
```

Execute o arquivo

Abra o terminal na pasta do arquivo. No Windows, use py app.py; se o comando disponível foi python, use python app.py. No macOS/Linux, use python3 app.py. Salve suas mudanças e execute novamente para ver o novo resultado.

Se o terminal disser que o arquivo não existe, confira a pasta e o nome. Se o comando de Python não for reconhecido, revise a instalação. O prompt interativo com >>> é diferente do terminal do sistema: os comandos de execução acima pertencem ao terminal.

PRATIQUE

Execute app.py e troque a mensagem. Não copie comandos do terminal para dentro do arquivo Python.

Leia e escreva suas instruções

Sintaxe é o conjunto de regras de escrita da linguagem. Parênteses, aspas, dois-pontos e espaços podem mudar o significado do código. Python diferencia letras maiúsculas e minúsculas: nome e Nome são identificadores diferentes.

Mensagens e comentários

```
01 | # Esta linha é um comentário
02 | print("Estou aprendendo Python.")
03 | print("Uma instrução por linha.")
```

Textos ficam entre aspas simples ou duplas. O símbolo # inicia um comentário fora de uma string: o restante daquela linha não é executado. Use comentários para explicar uma intenção que não esteja clara no código.

Indentação faz parte do código

```
01 | idade = 18
02 | if idade >= 18:
03 |     print("Você é maior de idade.")
04 | print("Consulta finalizada.")
```

Após os dois-pontos, o recuo agrupa as instruções do bloco. Neste guia, usamos quatro espaços por nível. A segunda mensagem está fora do bloco e aparece independentemente da condição. Não misture tabulações com espaços.

Não é necessário colocar ponto e vírgula no fim de cada linha. Ao copiar um bloco, preserve o recuo. O editor pode ajudar a inserir os espaços automaticamente.

PRATIQUE

No exemplo da idade, teste 17 e 18. Explique por que a mensagem “Consulta finalizada.” aparece nas duas execuções.

Variáveis e tipos básicos

Uma variável é um nome associado a um valor. A atribuição usa `=`. Você pode consultar esse nome e atribuir outro valor depois. Escolha nomes que indiquem o significado dos dados; evite espaços e não use palavras reservadas como `if` ou `for`.

```
01 | nome = "Ana"      # str: texto
02 | idade = 20        # int: inteiro
03 | altura = 1.68     # float: decimal
04 | estuda = True     # bool: verdadeiro ou falso
05 | print(nome, idade)
06 | print(type(altura))
```

Números decimais no código usam ponto, não vírgula. Os valores booleanos são `True` e `False`, com inicial maiúscula. `type()` informa o tipo; no exemplo, a última linha mostra `<class 'float'>`. O valor `None` representa ausência de valor em muitos contextos.

Atribuição e atualização

```
01 | pontos = 10
02 | pontos = pontos + 5
03 | print(pontos) # 15
```

O lado direito é calculado antes da atribuição. A forma `pontos += 5` também soma cinco ao valor atual. Uma variável deve receber um valor antes de ser usada. Não use nomes como `print`, `list` ou `input` para suas variáveis: eles escondem recursos da linguagem.

O texto `"20"` é diferente do número `20`. Python não transforma todo texto em número automaticamente. A escolha do tipo influencia as operações possíveis.

PRATIQUE

Crie variáveis para nome, idade e um indicador de estudante. Mostre os valores com `print()` e consulte o tipo de cada um.

Converse com quem usa o programa

`input()` apresenta uma mensagem, espera uma resposta e devolve uma string. Mesmo que a pessoa digite apenas números, você recebe texto. Para calcular, faça a conversão adequada.

```
01 nome = input("Seu nome: ").strip()
02 idade = int(input("Sua idade: "))
03 print(f"Olá, {nome}!")
04 print(f"No próximo ano: {idade + 1} anos.")
```

`strip()` devolve o texto sem espaços no início e no fim. `int()` converte uma representação de inteiro, como "20", em número. Se a pessoa digitar "vinte" ou "20.5", essa conversão gera `ValueError`. Vamos tratar esse caso na página 15.

Texto com valores: f-strings

O `f` antes das aspas permite inserir expressões entre chaves. É uma forma legível de montar mensagens. Você também pode mostrar valores separados por vírgulas com `print()`.

```
01 preco = float(input("Preço com ponto decimal: "))
02 print(f"Valor informado: R$ {preco:.2f}")
```

`float()` aceita um número decimal escrito com ponto, como 12.50. O formato `:.2f` mostra duas casas decimais; ele muda a apresentação, não transforma o valor em um cálculo financeiro exato. Neste exemplo, uma vírgula decimal precisa ser corrigida antes da conversão.

PRATIQUE

Peça o nome e uma cidade. Apresente uma frase com os dois valores usando uma f-string. Teste um nome com espaços nas extremidades.

Calcule e compare valores

Operadores permitem combinar e comparar dados. Parênteses ajudam a deixar explícita a ordem desejada. Multiplicação e divisão têm prioridade sobre soma e subtração.

```
01 | print(10 + 3)    # 13: soma
02 | print(10 - 3)    # 7: subtração
03 | print(10 * 3)    # 30: multiplicação
04 | print(10 / 3)    # aproximadamente 3.3333
05 | print(10 // 3)   # 3: divisão com piso
06 | print(10 % 3)    # 1: resto
07 | print(2 ** 3)    # 8: potência
```

A divisão / de inteiros produz float. A divisão // arredonda para baixo; por exemplo, `-7 // 2` é `-4`. % informa o resto e ajuda a verificar se um inteiro é par: `numero % 2 == 0`. Dividir por zero gera um erro.

Comparações devolvem booleanos

```
01 | idade = 18
02 | print(idade == 18) # True: igualdade
03 | print(idade != 18) # False: diferença
04 | print(idade >= 18) # True: maior ou igual
```

Use `<`, `<=`, `>` e `>=` para comparar ordem. `=` atribui; `==` compara. `and` exige que ambas as condições sejam verdadeiras; `or` aceita ao menos uma; `not` inverte o valor lógico.

```
01 | idade = 18
02 | tem_documento = True
03 | pode_entrar = idade >= 18 and tem_documento
04 | print(pode_entrar) # True
```

PRATIQUE

Calcule $(7 + 9) / 2$. Compare $7 + 9 / 2$ e explique a diferença. Teste se 12 e 13 são pares.

DECISÕES

09

Escolha caminhos com if

if executa um bloco quando sua condição é verdadeira. elif verifica outra condição se a anterior falhou. else atende os casos restantes. Em uma cadeia, apenas o primeiro bloco cuja condição é verdadeira é executado.

```
01 nota = 6.5
02 if nota >= 7:
03     print("Aprovado")
04 elif nota >= 5:
05     print("Recuperação")
06 else:
07     print("Reprovado")
```

Com 6.5, o programa mostra “Recuperação”. A ordem dos testes importa: se você testasse `nota >= 5` primeiro, uma nota 9 já entraria nesse caso. Os limites acima são apenas uma regra didática.

Valide também os limites

```
01 nota = 11
02 if nota < 0 or nota > 10:
03     print("Nota fora do intervalo.")
04 else:
05     print("Nota entre 0 e 10.")
```

Uma regra de negócio e uma validação têm funções diferentes. A regra classifica uma nota válida; a validação decide se o dado pode ser aceito. Combine os dois exemplos apenas depois de entender cada um.

PRATIQUE

Uma compra ganha frete grátis a partir de R\$ 100. Crie uma condição para mostrar “Frete grátis” ou “Frete cobrado”. Teste 99, 100 e 101.

Percorra valores com for

for percorre os elementos de uma sequência ou de outro objeto iterável. Você escreve uma vez o que deve acontecer com cada elemento. range() ajuda a gerar uma sequência de números inteiros.

```
01 | for numero in range(1, 4):  
02 |     print(numero)
```

Resultado: 1, 2 e 3, em linhas separadas. O início é incluído; o limite final é excluído. range(3) gera 0, 1 e 2. Você também pode definir o passo, como range(0, 6, 2), que gera 0, 2 e 4.

Some os valores de uma sequência

```
01 | total = 0  
02 | for numero in range(1, 6):  
03 |     total += numero  
04 | print(total) # 15
```

total começa em zero e acumula os números de 1 a 5. Observe que o último print está fora do bloco: ele mostra o resultado apenas depois que a repetição termina.

Percorra textos

```
01 | for letra in "Pato":  
02 |     print(letra)
```

Uma string também pode ser percorrida. Neste caso, cada repetição recebe um caractere. Na página 12, você fará algo parecido com uma lista de tarefas.

PRATIQUE

Use for e range() para mostrar a tabuada do 3, de 3 × 1 até 3 × 10. Mostre a operação e o resultado em cada linha.

Repita enquanto fizer sentido

`while` repete um bloco enquanto a condição for verdadeira. Use quando a quantidade de repetições depende de uma situação, como a pessoa escolher sair de um menu. A condição precisa ter uma forma de deixar de ser verdadeira.

```
01 | contador = 1
02 | while contador <= 3:
03 |     print(contador)
04 |     contador += 1
```

Sem o incremento, contador continuaria valendo 1 e o programa repetiria sem terminar. Se um programa no terminal ficar preso, Ctrl+C normalmente interrompe a execução.

Interrompa com `break`

```
01 | while True:
02 |     resposta = input("Digite sair para encerrar: ")
03 |     if resposta.strip().lower() == "sair":
04 |         break
05 |     print("Você ainda está no programa.")
06 |     print("Até a próxima!")
```

`while True` cria uma repetição que depende de uma saída explícita. `break` encerra o laço mais próximo. `lower()` devolve o texto em letras minúsculas; com `strip()`, facilita aceitar respostas como " SAIR ".

`continue` pula o restante da repetição atual e inicia a próxima. Use com cuidado: em um `while` com contador, pular a atualização pode impedir que o laço termine. O projeto final usa `break` para sair do menu.

PRATIQUE

Peça uma senha até a pessoa digitar "python". Ao acertar, mostre "Acesso liberado". É um exercício de repetição, sem segurança real de autenticação.

Organize dados em listas

Uma lista reúne valores em uma ordem. Os itens ficam entre colchetes e são separados por vírgulas. O primeiro índice é 0. `len()` informa quantos itens existem.

```
01 tarefas = ["Estudar", "Treinar"]
02 print(tarefas[0]) # Estudar
03 print(len(tarefas)) # 2
04 tarefas.append("Ler")
05 tarefas[1] = "Caminhar"
06 print(tarefas)
```

`append()` acrescenta ao final. A atribuição por índice substitui um item. A lista final será ["Estudar", "Caminhar", "Ler"]. Acessar um índice inexistente gera `IndexError`; verifique os limites antes de usar um número recebido da pessoa.

Liste e numere os itens

```
01 tarefas = ["Estudar", "Caminhar", "Ler"]
02 for numero, tarefa in enumerate(tarefas, start=1):
03     print(f"{numero}. {tarefa}")
```

`enumerate()` fornece um contador e o item em cada repetição. `start=1` muda o contador exibido; os índices reais da lista continuam começando em zero. Essa distinção será útil no projeto.

Remova um item pelo índice

```
01 tarefas = ["Estudar", "Caminhar", "Ler"]
02 removida = tarefas.pop(0)
03 print(removida) # Estudar
04 print(tarefas) # ["Caminhar", "Ler"]
```

`pop()` remove e devolve o item. Em uma lista vazia, `if not tarefas` é verdadeiro. Não remova itens da lista enquanto a percorre sem entender como os índices mudam.

PRATIQUE

Crie uma lista com três nomes. Adicione um quarto, remova o primeiro e mostre os nomes restantes com numeração.

Dê nomes aos dados com dicionários

Um dicionário associa chaves a valores. Ele ajuda a representar informações com significado: em vez de decorar a posição de um nome ou uma idade, você consulta a chave correspondente.

```
01 | pessoa = {"nome": "Ana", "idade": 20}
02 | print(pessoa["nome"]) # Ana
03 | pessoa["idade"] = 21
04 | pessoa["cidade"] = "Montes Claros"
05 | print(pessoa.get("telefone", "Não informado"))
```

As chaves deste exemplo são strings. Cada chave é única no dicionário. Colchetes consultam a chave e geram `KeyError` se ela não existir. `get()` permite fornecer um valor alternativo. Adicionar uma chave existente substitui seu valor.

Percorra chave e valor

```
01 | pessoa = {"nome": "Ana", "idade": 20}
02 | for chave, valor in pessoa.items():
03 |     print(f"{chave}: {valor}")
```

`items()` fornece pares de chave e valor. Listas e dicionários resolvem necessidades diferentes: uma lista representa uma sequência; um dicionário representa associações. Eles podem ser combinados.

```
01 | tarefas = [
02 |     {"titulo": "Estudar", "concluida": False},
03 |     {"titulo": "Ler", "concluida": True},
04 | ]
05 | print(tarefas[0]["titulo"]) # Estudar
```

PRATIQUE

Crie um dicionário com nome, preço e estoque de um produto. Mostre o nome, atualize o estoque e consulte uma chave ausente com `get()`.

Reutilize código com funções

Uma função organiza instruções com um nome. `def` inicia a definição; o bloco só é executado quando você chama a função. Parâmetros recebem dados da chamada; `return` devolve um resultado e encerra a execução da função.

```
01 def calcular_total(preco, quantidade):  
02     total = preco * quantidade  
03     return total  
04  
05 resultado = calcular_total(15, 4)  
06 print(resultado) # 60
```

`preco` e `quantidade` são parâmetros; 15 e 4 são argumentos da chamada. `total` é uma variável local da função. Cada chamada pode receber valores diferentes, mantendo a mesma regra.

Mostrar é diferente de devolver

```
01 def dobro(numero):  
02     return numero * 2  
03  
04 valor = dobro(5)  
05 print(valor + 1) # 11
```

`print()` apresenta algo na tela. `return` entrega um valor para o código que chamou a função. Uma função sem `return` explícito devolve `None`. Para continuar calculando com o resultado, normalmente você precisa devolvê-lo.

Responsabilidades pequenas

Escolha nomes como `listar_tarefas` ou `calcular_total`. Uma função deve ter uma responsabilidade fácil de explicar. A função pode receber uma lista e modificá-la; no projeto, `append()` e `pop()` alteram a lista passada como argumento.

PRATIQUE

Crie a função `tem_frete_gratis(total)`, que devolve `True` a partir de 100 e `False` abaixo disso. Teste com três valores.

Encontre a causa e trate a entrada

Erros fazem parte do desenvolvimento. Leia a última linha da mensagem para identificar o tipo e a descrição. No traceback, procure o arquivo e a linha do seu código envolvidos na falha. Corrija uma causa de cada vez e execute novamente.

SyntaxError: escrita inválida, como aspas não fechadas.

IndentationError: problema no recuo do bloco.

NameError: nome não definido.

TypeError: operação com tipos incompatíveis.

ValueError: valor inadequado para uma operação.

IndexError: índice fora da sequência.

Trate uma conversão esperada

```
01 entrada = input("Digite sua idade inteira: ")
02 try:
03     idade = int(entrada)
04 except ValueError:
05     print("Use um número inteiro, como 20.")
06 else:
07     if idade < 0:
08         print("A idade não pode ser negativa.")
09     else:
10         print(f"Idade informada: {idade}")
```

try delimita a operação que pode falhar. except ValueError atende essa falha específica. O else do try só roda se o bloco try terminou sem exceção. A validação de idade negativa continua necessária mesmo quando a conversão funciona.

Evite except sem tipo apenas para esconder qualquer problema. Use print() temporariamente para observar valores e type() para inspecionar tipos. Erros de lógica podem produzir respostas erradas sem mostrar uma exceção.

PRATIQUE

Execute o exemplo com 20, vinte e -1. Confira se cada entrada recebe uma resposta coerente e explique cada caminho.

Um organizador no terminal

Vamos reunir os conceitos em um programa que adiciona, lista e remove tarefas. O menu permanece aberto até você escolher sair. A lista começa vazia e existe somente durante a execução: ao fechar, os dados são perdidos. Salvar em arquivo será um próximo passo, fora deste projeto básico.

O que você vai construir

1. Adicionar uma tarefa sem aceitar texto vazio.
2. Listar tarefas com números começando em 1.
3. Remover uma tarefa usando o número exibido.
4. Tratar números inválidos e opções desconhecidas.
0. Encerrar o programa.

Monte um único arquivo

Crie tarefas.py. As páginas 17, 18 e 19 contêm três partes de um mesmo programa. Copie os blocos nessa ordem para o arquivo, mantendo os recuos. Execute somente depois de inserir todas as partes. Não inclua os números de linha do PDF.

```
01 | primeiro-python/  
02 | tarefas.py
```

A primeira parte define duas funções. A segunda define a remoção. A terceira cria a lista e chama essas funções de acordo com o menu. Nenhuma biblioteca externa é necessária.

Teste por comportamento

Comece listando uma lista vazia. Adicione duas tarefas e confira a ordem. Remova a segunda. Tente um número fora da lista, uma letra e um título só com espaços. Depois, escolha 0 e confirme que o programa termina.

PRATIQUE

Antes de escrever, explique por que o número 1 exibido para uma tarefa deve virar o índice 0 dentro da lista.

Adicionar e listar tarefas

```
01 def adicionar_tarefa(tarefas):
02     titulo = input("Nova tarefa: ").strip()
03     if not titulo:
04         print("Digite um título.")
05         return
06     tarefas.append(titulo)
07     print("Tarefa adicionada.")
08
09
10 def listar_tarefas(tarefas):
11     if not tarefas:
12         print("Nenhuma tarefa por aqui.")
13         return
14     for numero, titulo in enumerate(tarefas, start=1):
15         print(f"{numero}. {titulo}")
```

Entenda a entrada

`strip()` remove espaços das extremidades. Uma string vazia é considerada falsa em uma condição, então `if not titulo` recusa esse caso. `return` encerra a função antes de adicionar algo inválido.

Entenda a coleção

`tarefas.append(titulo)` altera a lista recebida. A função de listagem confere se há itens e usa `enumerate()` para mostrar números legíveis. Ela consulta a lista sem modificá-la.

Essas linhas apenas definem as funções. A chamada e a criação da lista aparecem na página 19. Continue no mesmo arquivo, após este bloco.

Remover com validação

```
01 def remover_tarefa(tarefas):
02     if not tarefas:
03         print("A lista está vazia.")
04         return
05     listar_tarefas(tarefas)
06     entrada = input("Número para remover: ")
07     try:
08         numero = int(entrada)
09     except ValueError:
10         print("Digite um número inteiro.")
11         return
12     if not 1 <= numero <= len(tarefas):
13         print("Número fora da lista.")
14         return
15     removida = tarefas.pop(numero - 1)
16     print(f"Removida: {removida}")
```

Proteja a operação

A função rejeita a lista vazia, tenta converter a entrada e verifica os limites. A comparação encadeada `1 <= numero <= len(tarefas)` exige que o número esteja entre 1 e a quantidade de tarefas, inclusive.

A pessoa escolhe pela numeração exibida. `numero - 1` converte esse número para o índice real. A validação acontece antes de `pop()`, evitando remover um item errado ou acessar uma posição inexistente.

Não é preciso um `else` depois de cada erro: o `return` já encerra a função. Continue acrescentando o bloco da próxima página ao mesmo arquivo.

Conecte tudo no menu

```
01 tarefas = []
02
03 while True:
04     print("\n1 Adicionar | 2 Listar | 3 Remover | 0 Sair")
05     opcao = input("Escolha: ").strip()
06     if opcao == "1":
07         adicionar_tarefa(tarefas)
08     elif opcao == "2":
09         listar_tarefas(tarefas)
10     elif opcao == "3":
11         remover_tarefa(tarefas)
12     elif opcao == "0":
13         print("Até a próxima!")
14         break
15     else:
16         print("Opção inválida.")
```

opcao permanece como string: para comparar escolhas de menu, não precisamos convertê-la em inteiro. O código chama uma das funções definidas anteriormente e depois volta ao início da repetição. O texto `\n` cria uma quebra de linha na mensagem.

Execute e confira

Na pasta do arquivo, use `py tarefas.py` no Windows ou o comando de Python 3 disponível no seu ambiente. Se uma função não for encontrada, confira se você inseriu as três partes na ordem certa.

Teste mínimo: liste vazio; adicione Estudar e Ler; remova 2; confirme que Estudar permaneceu. Na remoção, teste abc, 0 e 99. Digite uma opção desconhecida e verifique se o menu continua. Finalize com 0.

PRATIQUE

Melhoria: mostre a quantidade de tarefas no menu usando `len(tarefas)`. Confira o contador depois de cada adição e remoção.

EXERCÍCIOS

20

Teste o que você entendeu

Resolva sem consultar as respostas primeiro. Use um arquivo separado para cada exercício ou comente os trechos que não está testando. Execute e explique os resultados com suas palavras.

01 / Dados e entrada

Peça um nome e uma idade inteira. Mostre uma saudação e a idade no próximo ano. Qual é o tipo devolvido por `input()`? O que acontece se `int()` receber "vinte"?

02 / Cálculos

Use duas notas, 7 e 9, para calcular a média. Por que $(7 + 9) / 2$ é diferente de $7 + 9 / 2$?

03 / Decisões

Escreva `tem_frete_gratis(total)`, devolvendo um booleano. O frete é grátis quando total é maior ou igual a 100. Teste 99, 100 e 101.

04 / Repetições

Mostre os inteiros de 1 a 5 com `for` e `range()`. Depois, use um acumulador para mostrar a soma desses números.

05 / Coleções

Crie uma lista com três nomes. Adicione um quarto e remova o primeiro. Qual é o índice inicial da lista? Como você descobre quantos itens restaram?

06 / Evolua o projeto

Inclua no menu uma mensagem com a quantidade atual de tarefas. Depois, adicione a opção 4 para limpar a lista. Use `tarefas.clear()`. Essa ação deve funcionar também com a lista vazia.

PRATIQUE

Para cada exercício, teste o caso esperado e um caso de limite. Se consultar uma resposta, reescreva a solução sem olhar e explique por que funciona.

RESPOSTAS

21

Confira e compare

01. `input()` devolve `str`. `int("vinte")` gera `ValueError`; use o tratamento da página 15 se quiser responder a essa entrada sem encerrar o programa.

```
01 | nome = input("Nome: ").strip()
02 | idade = int(input("Idade inteira: "))
03 | print(f"Olá, {nome}! Próximo ano: {idade + 1}.")
```

02. A média é 8.0. Os parênteses fazem a soma acontecer primeiro; sem eles, `9 / 2` ocorre antes da soma e o resultado é 11.5.

```
01 | media = (7 + 9) / 2
02 | print(media) # 8.0
```

03. A comparação já devolve um booleano. Para 99, 100 e 101, os resultados são `False`, `True` e `True`.

```
01 | def tem_frete_gratis(total):
02 |     return total >= 100
```

04. `range(1, 6)` inclui 1 e exclui 6. A soma é 15.

```
01 | soma = 0
02 | for numero in range(1, 6):
03 |     print(numero)
04 |     soma += numero
05 | print(soma)
```

05. Use `append()` para adicionar e `pop(0)` para remover o primeiro. O índice inicial é 0 e `len(nomes)` devolve 3 depois dessas duas operações.

06. Dentro do `while`, mostre `print(f"Tarefas: {len(tarefas)}")`. Inclua 4 Limpar na mensagem do menu e, antes do `else final`, acrescente um `elif opcao == "4":` com `tarefas.clear()` e `print("Lista limpa.")` recuados dentro dele. Teste limpar zero e dois itens.

Glossário e referências

Algoritmo: sequência de passos para resolver uma tarefa.

Interpretador: programa que executa código Python.

Variável: nome associado a um valor.

Tipo: categoria de um valor, como str ou int.

String: sequência de caracteres.

Booleano: valor True ou False.

Condição: expressão que decide um caminho.

Laço: estrutura de repetição, como for ou while.

Índice: posição de um elemento em uma sequência.

Parâmetro: nome que recebe um argumento na função.

Exceção: sinal de uma falha durante a execução.

Próximos temas para estudar

Depois de praticar estes fundamentos, explore leitura e escrita de arquivos, módulos com import e testes automatizados. Para guardar as tarefas entre execuções, estude JSON e validação dos dados carregados. Bibliotecas externas, ambientes virtuais e aplicações web podem vir depois.

Referências oficiais

O texto e os exercícios deste guia foram organizados pela PatoDevs. Para aprofundar os detalhes da linguagem, consulte as referências abaixo. Links consultados em 5 de outubro de 2026.

[Python — Instalação e downloads](#)

[Python — Primeiros passos com números e textos](#)

[Python — Condições, repetições e funções](#)

[Python — Listas e dicionários](#)

[Python — Entrada, saída e formatação](#)

[Python — Erros e exceções](#)

Seu próximo passo é criar.

Você chegou ao fim deste guia e conheceu as primeiras ferramentas para programar em Python: dados, entrada e saída, decisões, repetições, coleções, funções e tratamento de erros.

Se acompanhou os exemplos e montou o organizador de tarefas, você já transformou uma ideia em um programa. O próximo passo é entender as escolhas que fez e experimentar mudanças próprias.

Não precisa saber tudo para começar.

Volte ao que ficou confuso. Teste um valor diferente, acompanhe o caminho de uma condição e observe como a lista muda. Encontrar um erro, consultar a documentação e reescrever uma função também fazem parte do aprendizado.

Seu próximo projeto pode ser uma calculadora, um quiz ou um organizador com novas opções. Escolha algo pequeno que consiga terminar. Quando funcionar, tente explicar cada parte sem depender do exemplo.

O aprendizado continua quando você coloca a mão no código.

Obrigado por aprender com a PatoDevs. Que este guia seja o começo dos programas que você ainda vai construir.